



INVENTIX · TEAM 4

Database build guide

How the Supabase database is created, in what order, the row level security that protects it, and the queries the backend runs against it.

Source docs/09-database-build.md

Companion 08-api-contract.md

Generated 2 September 2026

Contents

Why the database comes first

What can be built in parallel

Part 1 — Creating the Supabase project

Once, by one person

Running the SQL

Part 2 — Migration conventions

Part 3 — The tables, in build order

0001 profiles — §5.1

0002 product_catalog — §5.2

0003 supplier_listings — §5.3

0004 stock_items — §5.4

0005 orders — §5.5

0006 order_items — §5.6

0007 supplier_ratings — §5.7

0008 sales_uploads / 0009 sales_records — §5.8

0010 pos_product_aliases — §5.9

0011 stock_adjustments — §5.10

0012 seasonal_events — §5.11

0013 notifications / 0014 device_tokens — §5.12

0015 app_config — §12.1, §17

Part 4 — Indexes

Part 5 — Row level security

How it works

The role helper

The nine policy files

Verifying it actually works

Part 6 — The two SQL functions

apply_stock_adjustment.sql — §5.10

recompute_supplier_ranking.sql — §12, §14

Part 7 — Seeds

Part 8 — The queries the backend will actually run

The Stocks feed — one query, not N+1

The pie chart summary — derived, never counted separately

Supplier search by product — §9.1

Measured delivery speed — §12.1

Confirming an order — the check that must not be skipped

Matching an upload row to a catalog product — §6.6

Duplicate upload detection — §15.4

The grouped restock action — §6.4

Part 9 — Keeping the frontend in sync

Part 10 — Build checklist

How the Supabase database is created, in what order, why each decision was made, and the queries the backend will actually run against it.

Why the database comes first

Three reasons, in order of how much trouble each causes if ignored.

1. Row level security is the real access control, and reads bypass the backend entirely. Specification §3.1 lets the app read Supabase directly for display. The moment a real anon key goes into `frontend/.env`, every table is reachable from the phone. If policies are not already in place, any logged-in account can read every shop's stock levels and sales history — including suppliers, which §10.4 explicitly forbids. FastAPI cannot protect a read it is not part of.

This is not a step that can be done later. "Later" means the window in which the data was readable already happened.

2. Every `service.py` is queries against these tables. Writing service code first means inventing column names and renaming them afterwards.

3. The frontend already declares the schema. `frontend/src/types/database.ts` mirrors §5 table by table. Those interfaces are a contract — if a migration names a column differently nothing errors, the field simply arrives `undefined` on a screen, which is the most expensive kind of bug to find.

Specification §18 puts schema and policies first for the same reasons.

What can be built in parallel

`backend/app/domain/` touches neither HTTP nor SQL. The order state machine, ranking formula, message builder and threshold defaults are pure functions over plain data, and `backend/tests/` already contains exactly the four test files for them. A second person can build and fully test `domain/` before this database exists.

That is the natural split for a four-person team, and it answers open question Q9.

Part 1 — Creating the Supabase project

Once, by one person

1. Create a project at supabase.com. Choose the region closest to Sri Lanka — Singapore (`ap-southeast-1`) or Mumbai (`ap-south-1`). Region affects every request's latency and cannot be changed later without recreating the project.
2. Record the database password somewhere the team can reach. It is shown once.
3. From **Project Settings** → **API**, collect three values:

Value	Goes where	Safe to share?
Project URL	<code>frontend/.env</code> and <code>backend/.env</code>	Yes
<code>anon</code> public key	<code>frontend/.env</code> as <code>EXPO_PUBLIC_SUPABASE_ANON_KEY</code>	Yes — it is public by design and constrained by RLS
<code>service_role</code> key	<code>backend/.env</code> only	No. Never.

The `service_role` key bypasses row level security completely. In the React Native app it would be extractable from the installed APK in minutes, and would hand any user the entire database (\$15.3). It exists so the backend can write the `profiles.role` field that the client is forbidden to write.

Running the SQL

Everything in `database/` is plain SQL, run in order through the Supabase **SQL Editor**. There is no migration tool in this build, and none is needed for fifteen files — but the *order* matters, because foreign keys refer backwards.

1. `database/migrations/0001 ... 0015` in numeric order, no skipping
2. `database/functions/*.sql` they reference tables
3. `database/policies/*.sql` they reference tables and the role helper
4. `database/seeds/*.sql` `app_config` and `product_catalog` first

Running them out of order fails loudly on a missing table, which is the good case. Running policies before tables is the only genuinely confusing failure.

Part 2 — Migration conventions

A migration is **never edited after it has been run**. Changing the schema means adding `0016_something.sql`. This is not bureaucracy: it means the database can be rebuilt from zero at any time, every change has a date and an author in git, and no team member is ever running a schema that silently differs from someone else's.

One table per file, numbered in dependency order. `0005_orders.sql` can reference `0001_profiles.sql` because 1 runs before 5. This ordering is why the numbering exists at all.

Every table gets:

```
id          uuid primary key default gen_random_uuid()
created_at  timestampz not null default now()
updated_at  timestampz not null default now()
```

The exception is `profiles`, whose `id` is not generated — it must equal the Supabase auth user id, which is what ties a profile to a login.

`updated_at` needs a trigger to actually update; Postgres will not do it on its own. One trigger function, reused by every table.

Policies live in their own folder, not beside the tables. Row level security is the thing most likely to be wrong and hardest to notice — a missing policy is invisible until a supplier can read a shop's sales history. Keeping the nine policy files together means they can be reviewed as a set, against §15.1, without reading the schema at the same time.

Part 3 — The tables, in build order

Fifteen migrations, each implementing one section of §5. What follows is what each is *for* and the constraints that carry a rule — not a repeat of the column list, which is in the spec.

0001 profiles — §5.1

One row per user, both roles. `id` matches the Supabase auth user id.

The column that matters is `role`, which is `'customer'` or `'supplier'` and decides which navigation graph loads, which endpoints the caller may reach, and which policies apply. Constrain it with a `check`, not just a comment.

`delivery_areas text[]` is suppliers only. Postgres arrays are the right choice here over a join table: the list is short, never queried on its own, and only ever read whole.

The rule that must be enforced here: a client may not insert a profile, and may not update `role`. Both are covered in Part 5. Without them the backend's `/auth/profile` endpoint is decorative — anyone could grant themselves a supplier account with the anon key.

0002 product_catalog — §5.2

The single most important design decision in the schema. Both customer stock items and supplier listings point at catalog rows instead of storing free text.

If a customer typed "Rice 5kg" and a supplier typed "5kg rice bag", the application could never match them — and product search, supplier ranking, and sales-report matching all depend on both sides meaning the same row. Users pick from this table; they never type a product name.

`is_seasonal` marks products with festival demand spikes, used by §6.2.

0003 supplier_listings — §5.3

What a supplier offers. One row per catalog product per supplier — enforce that with a unique constraint on `(supplier_id, catalog_product_id)`, so a duplicate is refused by the database rather than by a service check someone might forget.

`is_active` retires a listing without deleting it, because `order_items` references listings and deleting one would break the history of every order that used it.

`quantity_available` is decremented when an order is confirmed, so a supplier is never shown as holding stock they have already committed.

0004 stock_items — \$5.4

A customer's own inventory. Unique on `(owner_id, catalog_product_id)` — one row per product per shop.

`restock_requested` `boolean` is the flag that prevents double ordering. It is true while an open order covers this item, and it is what turns a Low stock row into a Requested row that no longer opens the popup (§6.4).

`low_threshold` is per product because "low" genuinely differs: ten bags of rice may be low while two hundred packets of tea is normal.

0005 orders — \$5.5

One row per restock request to one supplier.

`status` must be constrained to the eight values in §11.1. The database refusing an unknown status is the last line of defence behind the state machine.

The stage timestamps are separate columns, not a log table: `requested_at`, `confirmed_at`, `rejected_at`, `processing_at`, `put_to_delivery_at`, `on_the_way_at`, `supplier_marked_delivered_at`, `purchased_at`. This looks redundant until you notice that the order detail screen needs the whole history in one row read, and that `purchased_at - requested_at` is the delivery-speed measurement behind 30 % of the supplier ranking. A log table would make both a join.

`supplier_marked_delivered_at` deserves its own note: it is set when a supplier says they have delivered, and it **does not change** `status`. The order stays `on_the_way` until the customer confirms (§11.3).

`auto_confirmed` `boolean` marks orders the system closed because the customer never confirmed, so they can be excluded from delivery-speed statistics — a supplier should not be credited with a fast delivery that nobody verified.

0006 order_items — \$5.6

The products inside an order; an order can cover several low items going to the same supplier.

`unit_price_at_order` captures the price at the moment of ordering. Without it, a supplier raising their price would silently rewrite the value of every historical order, and "total spend by supplier" in §7.1 would become fiction.

0007 supplier_ratings — \$5.7

One rating per completed order — unique on `order_id`. This is the **only** source of the quality component, which is 40 % of the ranking.

`quality_score` is 1–5; constrain the range.

0008 sales_uploads / 0009 sales_records — \$5.8

`file_hash` is the column that carries a rule: unique per customer, so re-uploading the same report is refused and stock is never decremented twice (§15.4).

`status` moves `pending` → `needs_mapping` → `applied`, or `failed`.

`sales_records.stock_item_id` is nullable, because a row whose product could not be matched still has to be recorded.

0010 pos_product_aliases — \$5.9

Maps a shop's POS export names to catalog products. Unique on `(customer_id, pos_product_name)`.

This is why mapping is a one-time cost: the owner matches "RICE-NADU-5KG" once and it resolves automatically on every upload afterwards. The unmatched list shrinks toward nothing.

0011 stock_adjustments — \$5.10

An audit row for every change to every quantity. \$5.10 calls this "not optional bookkeeping", and it is right: without it, a wrong quantity has no explanation and a bad upload cannot be reversed.

`change_quantity` is signed; `quantity_after` records the result, so the history can be read without replaying arithmetic. `reason` is one of `sales_upload`, `manual`, `order_received`, `damage`, `correction`. `source_id` points at the upload or order that caused it.

0012 seasonal_events — \$5.11

The festival calendar. `lead_time_months` is how far ahead to warn, `affected_categories text[]` is matched against catalog categories, `expected_uplift_pct` drives the suggested quantity.

`event_date` is per year and must be updated annually — Awurudu and Ramadan move.

0013 notifications / 0014 device_tokens — \$5.12

`related_order_id` and `related_stock_item_id` are both nullable and are what let a tap open the right screen.

`device_tokens.platform` is `'ios'` or `'android'` because Firebase needs to know which. `last_seen_at` lets a job expire stale tokens.

0015 app_config — \$12.1, \$17

Key-value configuration so tunable numbers are not literals in Python. A literal is not tunable; it is a redeploy.

What belongs here: the four ranking weights, the unanswered-order limit (default 24 hours), the auto-confirm delay (default 3 days), and the new-supplier order threshold (3).

Part 4 — Indexes

Postgres indexes primary keys and unique constraints automatically. It does **not** index foreign keys, and every query the backend runs filters on one. Add these with the tables, not after something feels slow.

```
create index on stock_items (owner_id);
create index on stock_items (owner_id, restock_requested);
create index on supplier_listings (supplier_id);
create index on supplier_listings (catalog_product_id) where is_active;
create index on orders (customer_id, status);
create index on orders (supplier_id, status);
create index on order_items (order_id);
create index on stock_adjustments (stock_item_id, created_at desc);
create index on sales_records (upload_id);
create index on pos_product_aliases (customer_id, pos_product_name);
create index on notifications (user_id, created_at desc) where read_at is null;
```

Two are worth explaining:

`supplier_listings (catalog_product_id) where is_active` is a **partial index**. Product search only ever looks at active listings, so indexing the inactive ones wastes space and slows writes.

`orders (customer_id, status)` and `(supplier_id, status)` are separate because the two feeds filter on different columns — the customer's Delivery feed and the supplier's Orders feed read the same table from opposite ends.

Part 5 — Row level security

The part most likely to be got wrong, and the part that matters most.

How it works

RLS is enforced by Postgres itself, on every query, regardless of who is asking or through what client. A policy is a condition automatically added to every statement.

Enable it on **every** table holding user data. A table with RLS enabled and no policy denies everything, which is the safe failure. A table with RLS *not* enabled is fully readable by anyone holding the anon key — and the anon key ships inside the app.

```
alter table stock_items enable row level security;
```

`auth.uid()` returns the caller's user id from their JWT. The `service_role` key bypasses all of this, which is exactly why it never leaves the backend.

The role helper

Several policies need to know whether the caller is a customer or a supplier. Written inline that becomes a subquery repeated across nine files, and repeated conditions drift.

```

create or replace function auth_role() returns text
language sql stable security definer set search_path = public as $$
    select role from profiles where id = auth.uid()
$$;

```

`security definer` is required: the function reads `profiles`, and without it the caller's own policy on `profiles` would apply recursively.

The nine policy files

File	Rule (§15.1)
<code>profiles.sql</code>	Read and update your own row. Any authenticated user may read a supplier's public business fields, because customers must browse suppliers. No client insert; role never updatable.
<code>stock_items.sql</code>	Readable and writable only by the owning customer. No supplier may read any stock item, ever.
<code>supplier_listings.sql</code>	Writable only by the owning supplier. Readable by any authenticated user, because customers must search listings.
<code>orders.sql</code>	Readable by the customer who placed it and the supplier who received it. Nobody else.
<code>order_items.sql</code>	Same visibility as the parent order, via an <code>exists</code> on <code>orders</code> .
<code>supplier_ratings.sql</code>	Writable only by the customer who placed the order being rated. Readable by that customer and the rated supplier. A supplier can never edit or delete a rating.
<code>sales_data.sql</code>	<code>sales_uploads</code> , <code>sales_records</code> , <code>stock_adjustments</code> , <code>pos_product_aliases</code> — owning customer only. Never visible to any supplier.
<code>notifications.sql</code>	Your own only.
<code>catalog.sql</code>	Everyone reads. Nobody writes except the backend.

The shapes look like this:

```
-- stock_items: the simple case
create policy "own stock" on stock_items
  for all using (owner_id = auth.uid()) with check (owner_id = auth.uid());

-- orders: two parties, one row
create policy "orders visible to both parties" on orders
  for select using (customer_id = auth.uid() or supplier_id = auth.uid());

-- order_items: inherits from the parent
create policy "order items follow the order" on order_items
  for select using (exists (
    select 1 from orders o where o.id = order_items.order_id
    and (o.customer_id = auth.uid() or o.supplier_id = auth.uid())
  ));

-- profiles: the one that protects role
create policy "read own profile" on profiles
  for select using (id = auth.uid());
create policy "read supplier public fields" on profiles
  for select using (role = 'supplier');
create policy "update own profile, not role" on profiles
  for update using (id = auth.uid())
  with check (id = auth.uid() and role = (select role from profiles where id = auth.uid()));
```

Note the difference between `using` and `with check`: `using` decides which rows you may see or touch, `with check` decides what the row is allowed to look like afterwards. The `profiles` update policy needs both — `using` to stop you editing someone else's row, `with check` to stop you changing your own `role`.

There is **no insert policy on `profiles`**. That absence is deliberate and load-bearing: it is what forces registration through `POST /auth/profile`, where the backend sets the role with the service key.

Verifying it actually works

A policy that is written but wrong looks identical to one that is right. Test it explicitly, before any real data exists:

1. Register two customers, A and B, and one supplier S.
2. Sign in as A in the app and add a stock item.
3. Sign in as B and query `stock_items` directly through the Supabase client. **You must get zero rows**, not an error — RLS filters, it does not throw.
4. Sign in as S and query `stock_items` and `sales_records`. Zero rows both times.
5. As B, attempt `update profiles set role = 'supplier' where id = <B>`. It must be refused.
6. As S, attempt to update one of A's listings. Refused.

Step 5 is the one people skip. It is also the one that matters most: a customer who can promote themselves to supplier can read every listing and every order in the system.

Repeat this whenever a policy file changes. It takes five minutes and it is the only proof.

Part 6 — The two SQL functions

Some work must be atomic in a way an application cannot guarantee across two round trips.

`apply_stock_adjustment.sql` — §5.10

Changes a quantity and writes its audit row **in one statement**. Every quantity change in the application goes through it: manual adjustments, sales uploads, delivery receipts.

```
create or replace function apply_stock_adjustment(
    p_stock_item_id uuid, p_change integer, p_reason text,
    p_source_id uuid, p_created_by uuid
) returns integer
language plpgsql as $$
declare v_after integer;
begin
    update stock_items
        set quantity_on_hand = quantity_on_hand + p_change,
            last_counted_at = now()
    where id = p_stock_item_id
    returning quantity_on_hand into v_after;

    if v_after is null then raise exception 'stock item not found'; end if;
    if v_after < 0 then raise exception 'quantity cannot go below zero'; end if;

    insert into stock_adjustments
        (stock_item_id, change_quantity, quantity_after, reason, source_id, created_by)
    values (p_stock_item_id, p_change, v_after, p_reason, p_source_id, p_created_by);

    return v_after;
end $$;
```

Why a function rather than two queries from Python: between an `update` and an `insert` a process can crash, a connection can drop, or a second request can interleave. Any of those leaves a quantity with no explanation — the exact situation §5.10 exists to prevent. Here it is one transaction: both happen or neither does.

The negative check lives here too, so no caller can bypass it.

`recompute_supplier_ranking.sql` — §12, §14

Recomputes every supplier's score, run daily by `jobs/ranking_recalc.py`.

It computes the two context-free components:

- **Quality** — average `quality_score` , **weighted by how many ratings exist**, so one five-star rating does not outrank fifty four-star ones (§12.2).
- **Delivery speed** — average `purchased_at - requested_at` across completed orders, excluding `auto_confirmed` ones, compared against other suppliers.

Suppliers below the configured minimum of completed orders get a **neutral default, not zero**. Scoring a missing rating as zero would rank every new supplier last permanently, so they could never earn the first order that would give them a rating (§12.2). This is a real failure mode, not a hypothetical.

Availability and price — the other 30 % — are **not** computed here, because they depend on what the customer is currently ordering. They are applied per query in `backend/app/domain/ranking.py` . That split is why the profile screen shows a score built from 70 % of the formula, renormalised.

Doing this in SQL rather than Python is deliberate: it is an aggregate over every order and rating in the system, and pulling that into the application to average it would be slow and pointless.

Part 7 — Seeds

File	Contents	Why it matters
<code>app_config.sql</code>	Ranking weights 40/30/20/10, unanswered limit 24 h, auto-confirm 3 days, new-supplier threshold 3	Missing rows mean every tunable reads as null and jobs misbehave
<code>product_catalog.sql</code>	Common Sri Lankan grocery products	An empty catalog means nobody can add a stock item or a listing — the app is unusable from the first screen
<code>seasonal_events.sql</code>	Awurudu, Christmas, Ramadan, Vesak with dates and uplift	The smart dashboard has nothing to warn about without it
<code>demo_data.sql</code>	A populated shop, suppliers, orders	For demonstrations. Never run against real data

`app_config` and `product_catalog` are not optional extras — they are required for a fresh install to function at all. Open question Q1 covers who curates the catalog long term.

Part 8 — The queries the backend will actually run

The ones worth writing down, because they are the ones easy to get subtly wrong.

The Stocks feed — one query, not N+1

The home screen needs every stock item with its catalog product and its preferred supplier's name. The naive version fetches stock items and then loops, making one request per row.

```

select s.id, s.quantity_on_hand, s.low_threshold, s.restock_requested,
       s.preferred_supplier_id,
       row_to_json(c)           as product,
       sup.business_name       as preferred_supplier_name,
       l.unit_price
from stock_items s
join product_catalog c  on c.id = s.catalog_product_id
left join profiles sup  on sup.id = s.preferred_supplier_id
left join supplier_listings l
      on l.supplier_id = s.preferred_supplier_id
      and l.catalog_product_id = s.catalog_product_id
      and l.is_active
where s.owner_id = :customer_id
order by c.name;

```

Both joins to the supplier are `left`: a stock item with no preferred supplier is normal and the row must still be returned. An inner join here silently hides products, which presents as "some of my stock is missing" and is genuinely hard to trace.

`status` is derived afterwards in `domain/stock.py`, not in SQL, so the classification rule stays in one place.

The pie chart summary — derived, never counted separately

```

select count(*) as total,
       count(*) filter (where quantity_on_hand > low_threshold) as in_stock,
       count(*) filter (where quantity_on_hand <= low_threshold
                        and not restock_requested)           as low_stock,
       count(*) filter (where quantity_on_hand <= low_threshold
                        and restock_requested)                as restock_requested
from stock_items where owner_id = :customer_id;

```

These three conditions must be the same three in `domain/stock.py`. If they drift, the chart says four items are low while the list below shows three — and both look correct in isolation.

Supplier search by product — §9.1

```
select p.id, p.business_name, p.city, p.is_active,
       l.unit_price, l.quantity_available, l.lead_time_days, l.min_order_quantity,
       r.score, r.average_rating, r.rating_count, r.measured_delivery_days
from supplier_listings l
join profiles p on p.id = l.supplier_id
left join supplier_ranking r on r.supplier_id = p.id
where l.catalog_product_id = :product_id
      and l.is_active and p.is_active
order by r.score desc nulls last;
```

`nulls last` matters. A supplier the ranking job has not scored yet must appear at the bottom, and Postgres sorts nulls first by default in a descending order.

`supplier_ranking` is the precomputed table the daily job writes.

Measured delivery speed — §12.1

```
select supplier_id,
       avg(extract(epoch from (purchased_at - requested_at)) / 86400) as days,
       count(*) as completed
from orders
where status = 'purchased' and not auto_confirmed
group by supplier_id;
```

`not auto_confirmed` is the important clause. An order the system closed after three days of silence says nothing about how fast that supplier delivered, and counting it would credit them for a delivery nobody verified.

Confirming an order — the check that must not be skipped

```
select l.id, l.quantity_available, oi.quantity_requested, c.name
from order_items oi
join supplier_listings l on l.id = oi.listing_id
join product_catalog c on c.id = oi.catalog_product_id
where oi.order_id = :order_id
      and l.quantity_available < oi.quantity_requested;
```

Any row returned means the supplier can no longer fill the order, and Confirm must be refused with that product named (§10.2). Availability may have fallen since the order was placed; the check at order time does not carry forward.

Matching an upload row to a catalog product — §6.6

```
select catalog_product_id from pos_product_aliases
where customer_id = :customer_id and pos_product_name = :name;
```

Exact match, deliberately. Fuzzy matching would occasionally map "RICE-NADU-5KG" to the wrong product, and a wrong match silently decrements the wrong stock item — worse than asking the owner once and remembering the answer forever.

Duplicate upload detection — §15.4

```
select id from sales_uploads
where customer_id = :customer_id and file_hash = :hash;
```

A row means reject. Back it with a unique constraint on `(customer_id, file_hash)` so a race between two simultaneous uploads cannot slip both through.

The grouped restock action — §6.4

```
select s.preferred_supplier_id, sup.business_name, count(*) as items
from stock_items s
join profiles sup on sup.id = s.preferred_supplier_id
where s.owner_id = :customer_id
and s.quantity_on_hand <= s.low_threshold
and not s.restock_requested
group by s.preferred_supplier_id, sup.business_name
having count(*) > 1;
```

`having count(*) > 1` is what makes it a *grouped* action. One low item from a supplier is a normal row, not "order 1 item from Lanka Traders".

This currently runs in the app. It should move here.

Part 9 — Keeping the frontend in sync

`frontend/src/types/database.ts` mirrors these tables and was written from §5 before the schema existed. After every migration, check the two still agree.

Supabase can generate the types directly:

```
npx supabase gen types typescript --project-id <id> > src/types/database.ts
```

The generated file will not carry the section comments the hand-written one has, so either keep generating and lose them, or diff the generated output against the hand-written file and apply the differences. Either is

fine. Letting them drift is not — a renamed column produces `undefined` on a screen with no error anywhere.

Part 10 — Build checklist

#	Step	Done when
1	Supabase project created, keys distributed	<code>service_role</code> is in <code>backend/.env</code> only
2	Migrations 0001–0015 run in order	Every table visible in the Table Editor
3	Indexes added	All of Part 4 applied
4	<code>apply_stock_adjustment</code> and <code>recompute_supplier_ranking</code> created	Both callable from the SQL editor
5	RLS enabled on every user-data table	No table shows "Unrestricted" in Supabase
6	Nine policy files applied	—
7	RLS verified by the Part 5 procedure	B cannot read A's stock; B cannot promote themselves
8	<code>app_config</code> and <code>product_catalog</code> seeded	Product search returns results
9	<code>seasonal_events</code> seeded	Dashboard shows a warning card
10	<code>frontend/src/types/database.ts</code> checked against the schema	Column names match exactly

Step 7 is the one not to skip. Steps 1–6 can all be done correctly and still leave the data readable by the wrong people. Step 7 is the only thing that proves otherwise.